Fall 2024 Date: Sep 10, 2024

Artificial Intelligence

Lab 4 - Sec A

Instructor: Husnain Haider
Lab Instructors:
Rimsha Aamir
Muhammad Taha
Abdul Rehman
Moeed Ahmad
Muhammad Ahmad

Python Lab: Introduction to Graphs and Trees

What are Graphs?

A graph is a data structure that consists of a set of nodes (also called vertices) and a set of edges that connect pairs of nodes. Graphs are widely used to represent various real-world relationships, such as social networks, transportation networks, and web page linking structures.

Vertices (Nodes): The fundamental units of a graph, representing entities.
Edges: The connections between vertices that define the relationships between them.

Types of Graphs:

- Directed Graph: A graph where edges have a direction, pointing from one vertex to another.
- Undirected Graph: A graph where edges have no direction; connections are mutual.
- Weighted Graph: A graph where edges have weights or costs associated with them.

What are Trees?

A tree is a special type of graph that is connected and acyclic (contains no cycles). It is a hierarchical data structure that consists of nodes connected by edges, resembling a tree-like structure. Trees are used in various applications, including file systems, organizational structures, and decision-making processes.

- Root Node: The top node of the tree, which has no parent.
- Child Node: A node that descends from another node.
- Leaf Node: A node with no children.
- Subtree: A tree formed by a node and its descendants.
- Binary Tree: A tree in which each node has at most two children (left and right).

How Graphs and Trees are Implemented?

Usually we follow two data structures: Adjacency List and Adjacency Matrix.

1. Adjacency List:

An adjacency list is a way to represent a graph using a collection of lists or dictionaries. Each node has a list of nodes it is directly connected to (its neighbors).

Structure: A dictionary where keys are nodes, and values are lists of adjacent nodes.

Example:
Graph: 0 -- 1, 1 -- 2, 1 -- 3

Adjacency List Representation:

```
graph = {
    0: [1],
    1: [0, 2, 3],
    2: [1],
    3: [1]
}
```

2. Adjacency Matrix:

An adjacency matrix is a 2D array (or list of lists) used to represent a graph. Each cell in the matrix indicates whether there is an edge between two nodes.
Structure: A 2D list where `matrix[i][j]` is 1 if there is an edge between node `i` and node `j`, and 0 otherwise.

- Example:
Graph: 0 -- 1, 1 -- 2, 1 -- 3
Adjacency Matrix Representation:

```
matrix = [
    [0, 1, 0, 0],  # Node 0
    [1, 0, 1, 1],  # Node 1
    [0, 1, 0, 0],  # Node 2
    [0, 1, 0, 0]   # Node 3
]
```

Pseudocode for Both Representations:

Adjacency List Pseudocode:

```
# Create an empty adjacency list
graph = {}

# Function to add an edge between node u and node v
def add_edge(u, v):
    if u not in graph:
        graph[u] = []
    if v not in graph:
        graph[v] = []
    graph[u].append(v)
    graph[v].append(u)
```

Adjacency Matrix Pseudocode:

```
# Initialize an empty n x n adjacency matrix with 0s
n = 4  # Number of nodes
matrix = [[0 for _ in range(n)] for _ in range(n)]


# Function to add an edge between node u and node v
def add_edge(u, v):
    matrix[u][v] = 1
    matrix[v][u] = 1
```

Lab Tasks

Task 1: Implement a Simple Undirected Graph

Create a Python class named Graph that represents an undirected graph. The graph should allow adding nodes and edges between them. The main functionality should include:

- add_node(node): Adds a node to the graph.
- add_edge(node1, node2): Adds an undirected edge between two nodes.
- display(): Prints the graph as an adjacency list showing each node and its connected nodes.

Task 1 Help:

```
class Graph:
    def __init__(self):
        # Initialize an empty dictionary to store the graph

    def add_node(self, node):
        # Add a node to the graph if it does not already exist

    def add_edge(self, node1, node2):
        # Add an undirected edge between node1 and node2

    def display(self):
        # Display the graph as an adjacency list
```

Example

```
g = Graph()
g.add_node('A')
g.add_node('B')
g.add_node('C')
g.add_edge('A', 'B')
g.add_edge('A', 'C')
g.add_edge('B', 'C')
g.display()
```

Output

```
A: B, C
B: A, C
C: A, B
```

Task 2: Implement a Simple Tree Structure

Create a Python class named TreeNode that represents a node in a tree, and another class named Tree to manage the entire tree structure. Each node should store a value and a list of its children. The main functionality should include:

- add_child(parent, child): Adds a child node to a parent node.
- display(): Prints the tree structure with each node and its children.

Task 2 Help:

```
class TreeNode:
    def __init__(self, value):
 # Initialize a tree node with a value and an empty list of children
        self.value = value
        self.children = []

    def add_child(self, child_node):
        # Add a child node to the current node
        self.children.append(child_node)



class Tree:
    def __init__(self, root):
        # Initialize the tree with a root node

    def add_child(self, parent_value, child_value):
        # Adds a child node to the tree under the given parent node

    def find_node(self, node, value):
        # Recursively search for a node with the specified value

    def display(self, node=None, level=0):
        # Display the tree structure starting from the given node


# Example Output
root = TreeNode('Root')
```

```
tree = Tree(root)
tree.add_child('Root', 'Child1')
tree.add_child('Root', 'Child2')
tree.add_child('Child1', 'Grandchild1')
tree.display()
```

```
Output
Root
    Child1
        Grandchild1
    Child2
```

Task 3: Represent a Graph with an Adjacency Matrix

Implement an undirected graph using an adjacency matrix. The adjacency matrix should be a 2D list (matrix) where each cell represents whether an edge exists between two nodes. Implement the following functionalities:

1. Add nodes to the graph and update the matrix size accordingly.
2. Add edges between nodes by updating the matrix cells.
3. Display the adjacency matrix.

Hints:
- Initialize the matrix with zeros and update values to 1 when an edge is added.
- Ensure that the matrix remains symmetric for an undirected graph.

Task 3 Help:

```
class Graph:
    """Class to represent an undirected graph using an adjacency matrix."""
    def __init__(self):
        self.nodes = []  # List to keep track of node labels
        self.matrix = []  # 2D list to represent the adjacency matrix

    def add_node(self, label):
        """Adds a new node to the graph and updates the adjacency matrix size."""

    def add_edge(self, node1, node2):
```

```
        """Adds an edge between two nodes by updating the adjacency matrix."""

    def display_matrix(self):
        """Displays the adjacency matrix."""
```

Example:

```
# Create a graph
graph = Graph()

# Add nodes to the graph
graph.add_node("A")
graph.add_node("B")
graph.add_node("C")
graph.add_node("D")

# Add edges between nodes
graph.add_edge("A", "B")
graph.add_edge("A", "C")
graph.add_edge("B", "D")
graph.add_edge("C", "D")

# Display the adjacency matrix
graph.display_matrix()
```

Output:

```
Adjacency Matrix:
   A  B  C  D
A [0, 1, 1, 0]
B [1, 0, 0, 1]
C [1, 0, 0, 1]
D [0, 1, 1, 0]
```